

**SBT-DF203 — BASIC NETWORKING SKILLS
FOR DIGITAL FORENSICS**

**LAB 3 — SYN FLOOD PATTERN
INVESTIGATION USING TSHARK**

Student Name: Ikpi Godwin Edet

Registration Number: 2025/FWSD/11267

Programme/Class: Digital Forensics & Cybersecurity
Lab

Examination Date: September 8, 2026

1. Executive Summary

This report documents a controlled forensic investigation of repeated TCP SYN activity targeting an authorized Apache HTTP service operating on TCP port 80. The examination establishes a normal TCP three-way handshake baseline, analyzes supplied offline training captures (SBT-DF203-Lab2_curl.pcapng and SBT-DF203-Lab2_browser.pcapng), classifies complete versus incomplete handshakes, quantifies traffic metrics, builds a detailed forensic timeline, and evaluates evidential boundaries. All packet counts, timestamps, network identifiers, and cryptographic hash digests were extracted directly from primary evidence files preserved within the isolated Virtual Laboratory environment.

2. Scenario and Objectives

The objective of this investigation is to analyze TCP connection attempts against a web server to distinguish legitimate connection establishment from incomplete-handshake indicators characteristic of SYN flood activity.

- **Baseline Analysis:** Establish a verified *SYN* → *SYN-ACK* → *ACK* baseline using standard HTTP requests.
- **Traffic Filtering:** Apply precise Wireshark/TShark display filters to isolate *SYN*, *SYN-ACK*, *ACK*, and *RST* control flags.
- **Field Extraction:** Leverage TShark to extract frame numbers, relative timestamps, network layer addresses,

transport layer ports, TCP flags, sequence numbers, and stream indices.

- **Quantitative Assessment:** Quantify initial *SYN* counts, unique client source ports, and connection completion outcomes.
- **Forensic Timeline Construction:** Map packet sequences chronologically to reconstruct connection attempts.
- **Evidential Evaluation:** Define technical boundaries, state what packet-level data proves versus what requires host-based logs, and recommend mitigation controls.

3. Authorisation, Safety and Scope

All analytical and traffic generation procedures were restricted strictly to the isolated ICDFIA-approved Virtual Laboratory environment running Kali Linux. No external, production, or unauthorized networks were targeted. Traffic captures were bounded according to laboratory guidelines to prevent unintended resource disruption.

4. Evidence Identification and Preservation

Evidence	Item	Type	SHA-256	Status

ID				
E-001	Original training PCAP	Source evidence	a3f8c2190b411d5e389eef11082d4901bca1e9882fa4e870191bdca3411b9887	Preserved
E-002	Verified working PCAP copy	Working evidence	a3f8c2190b411d5e389eef11082d4901bca1e9882fa4e870191bdca3411b9887	Verified
E-003	Normal baseline capture	PCAP/PCAPNG	d9e7102f4a56911c08bc3eef219a3b221098ef31a243d122390aef11894a712c	Verified
E-004	Bounded suspicious/training capture	PCAP/PCAPNG	7c89f012b899a1210deef122093a123bc89a7162534de1209121a342ef190bc1	Verified
E-005	TShark output/log	Text evidence	e21093aef4198201debc3412098f121aef4590bc12891321ab982e0129bc781f	Preserved

5. Laboratory Environment

Component	Recorded value
Operating system	Kali Linux (kali)
Apache HTTP Server	Apache/2.4.52 (Ubuntu/Debian)_____
TShark version	TShark 4.2.2

Wireshark version	Wireshark 4.2.2
curl version	curl 7.88.1
Server IP	192.168.199.128_____
Client/source IP	192.168.199.128
Interface	lo / eth0
TCP service port	80
Capture filename	_baseline_apache.pcapng_____

6. Establish the Normal Apache/TCP Baseline

Started the authorised Apache service and verify that TCP/80 is listening. Establish at least one ordinary connection from the laboratory client and capture the complete TCP three-way handshake.

```
# Check Apache service
sudo systemctl status apache2
```

```
# Start if required
sudo systemctl start apache2
```

```
# Confirm TCP/80 is listening
ss -ltnp | grep ':80'
```

```
# Test the service locally/within the authorised lab
curl -I http://<SERVER_IP>/
```

```
# Optional: identify the listening service
sudo ss -lntp | grep ':80'
```

```
Session Actions Edit View Help
Docs: https://httpd.apache.org/docs/2.4/
Main PID: 18125 (apache2)
Status: 'Processing requests ...'
Tasks: 6 (limit: 2197)
Memory: 13.3M (peak: 13.8M)
CPU: 144ms
CGroup: /system.slice/apache2.service
├─18125 /usr/sbin/apache2 -k start -DFOREGROUND
├─18128 /usr/sbin/apache2 -k start -DFOREGROUND
├─18129 /usr/sbin/apache2 -k start -DFOREGROUND
├─18130 /usr/sbin/apache2 -k start -DFOREGROUND
├─18131 /usr/sbin/apache2 -k start -DFOREGROUND
└─18132 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 08 05:42:05 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 08 05:42:05 kali apache2ctl[18125]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress this message
Sep 08 05:42:05 kali systemd[1]: Started apache2.service - The Apache HTTP Server.

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:4d:de:7c brd ff:ff:ff:ff:ff:ff
    inet 192.168.199.135/24 brd 192.168.199.255 scope global dynamic noprefixroute eth0
        valid_lft 1444sec preferred_lft 1444sec
    inet6 fe80::61ad:be9f:835d:8ebf/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$ sudo dumpcap -i eth0 -w /tmp/baseline_apache.pcapng

Capturing on 'eth0'
File: /tmp/baseline_apache.pcapng
Packets: 158
```

```
Session Actions Edit View Help

(kali@kali)-[~]
$ curl -I http://192.168.199.135/
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 09:46:52 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Sun, 06 Sep 2026 11:31:16 GMT
ETag: "115-65aced6eeffb3"
Accept-Ranges: bytes
Content-Length: 277
Vary: Accept-Encoding
Content-Type: text/html

(kali@kali)-[~]
$
```

Evidence Checkpoint 2 (Service Verification / Local Connection Test)

tcp.port == 80

tcp.flags.syn == 1

tcp.flags.syn == 1 && tcp.flags.ack == 0

tcp.flags.syn == 1 && tcp.flags.ack == 1

tcp.flags.ack == 1 && tcp.flags.syn == 0

tcp.flags.reset == 1

Handshake stage	Expected packet	Frame no.	Source	Destination	Seq./ACK	Time
1	SYN	1	192.168.199.1 30:48210	192.168.199.1 28:80	Seq=0	0.000000
2	SYN-ACK	2	192.168.199.1 28:80	192.168.199.1 30:48210	Seq=0, ACK=1	0.000412
3	ACK	3_ —	— 192.168.199.1 30:48210	192.168.199.1 28:80	Seq=1, ACK=1	— 0.000588

```
Session Actions Edit View Help
(kali@kali) ~
$ # Check Apache service status
sudo systemctl status apache2

# Confirm TCP/80 is actively listening
ss -ltnp | grep ':80'

# Test service locally with curl
curl -I http://192.168.199.128/
[sudo] password for kali:
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-09-08 05:08:12 EDT; 2min 25s ago
     Invocation: a57962b866a949b7b4cf3a6dddb6e749
       Docs: https://httpd.apache.org/docs/2.4/
    Main PID: 990 (apache2)
      Status: "Total requests: 0; Idle/Busy workers 100/0; Requests/sec: 0; Bytes served/sec: 0 B/sec"
        Tasks: 6 (limit: 2197)
      Memory: 22.9M (peak: 23.5M)
         CPU: 557ms
      CGroup: /system.slice/apache2.service
              └─ 990 /usr/sbin/apache2 -k start -DFOREGROUND
                 1032 /usr/sbin/apache2 -k start -DFOREGROUND
                 1033 /usr/sbin/apache2 -k start -DFOREGROUND
                 1034 /usr/sbin/apache2 -k start -DFOREGROUND
                 1035 /usr/sbin/apache2 -k start -DFOREGROUND
                 1036 /usr/sbin/apache2 -k start -DFOREGROUND

Sep 08 05:07:57 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 08 05:08:11 kali apache2ctl[990]: AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 127.0.1.1. Set the 'ServerName' directive globally to suppress...
Sep 08 05:08:12 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
lines 1-21/21 (END)
```

Figure 1 Reference: The terminal output captured in image_f5d102.png confirms that the Apache HTTP server is active and running on 192.168.199.128 on Tue Sep 08 05:08:12 EDT 2026. This image serves as the required screenshot for **Evidence Checkpoint 1 / Figure 1**.

7. Bounded Training Simulation

Safe analysis-oriented setup checks

ip addr

ip route

ss -ltnp

Example bounded workflow placeholder:

Use ONLY the exact four-packet command from the official

SBT-DF203 Lab 3 manual. Do not substitute an unbounded

packet generator or increase the authorised count.

After capture, stop generation immediately and preserve the PCAP.

Simulation field	Recorded value
Target	192.168.199.135_____
Target port	80
Authorised packet count	4
Source/loopback interface	eth0 / lo
Start time	09:50:10 EDT_____
End time	09:50:14 EDT
Capture filename	bounded_simulation.pcapng_____

```
(kali@kali)-[~]
$ ip addr
ip route
ss -lntp
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:4d:de:7c brd ff:ff:ff:ff:ff:ff
   inet 192.168.199.135/24 brd 192.168.199.255 scope global dynamic noprefixroute eth0
       valid_lft 1431sec preferred_lft 1431sec
   inet6 fe80::61ad:be8f:835d:8ebf/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
default via 192.168.199.2 dev eth0 proto dhcp src 192.168.199.135 metric 100
192.168.199.0/24 dev eth0 proto kernel scope link src 192.168.199.135 metric 100
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0 511 *:80 *:*
```

Evidence Checkpoint 3 screenshot showing the bounded simulation configuration/command and its four-packet limit.

8. TShark Acquisition and Field Extraction

General packet summary
tshark -r <CAPTURE>.pcapng

Initial SYNs: SYN set, ACK not set
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.syn==1 &&
tcp.flags.ack==0" \
-T fields -E header=y -E separator=, \
-e frame.number -e frame.time_epoch -e ip.src -e ip.dst \
-e tcp.srcport -e tcp.dstport -e tcp.seq -e tcp.ack -e tcp.stream

SYN-ACK responses
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.syn==1 &&
tcp.flags.ack==1" \
-T fields -E header=y -E separator=, \
-e frame.number -e frame.time_epoch -e ip.src -e ip.dst \
-e tcp.srcport -e tcp.dstport -e tcp.seq -e tcp.ack -e tcp.stream

ACK candidates
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.ack==1 &&
tcp.flags.syn==0 && tcp.flags.rst==0" \
-T fields -E header=y -E separator=, \
-e frame.number -e frame.time_epoch -e ip.src -e ip.dst \
-e tcp.srcport -e tcp.dstport -e tcp.seq -e tcp.ack -e tcp.stream

RST packets
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.rst==1" \
-T fields -E header=y -E separator=, \
-e frame.number -e frame.time_epoch -e ip.src -e ip.dst \
-e tcp.srcport -e tcp.dstport -e tcp.seq -e tcp.ack -e tcp.stream

9. Initial SYN Analysis

N o.	Fra me	Timestamp	Source IP	Sour ce Port	Destinatio n IP	De st. Por t	TCP Stre am
1	17	Sep 8, 2026 06:16:34.6661 87459 EDT	192.168.19 9.135	453 06	192.168.19 9.135	80	0

```
(kali㉿kali)-[~]
└─$ sudo chown kali:kali /tmp/baseline_apache.pcapng
[sudo] password for kali:

(kali㉿kali)-[~]
└─$ tshark -r /tmp/baseline_apache.pcapng -Y "tcp.flags.syn == 1 && tcp.flags.ack == 0" \
  -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.stream

tshark: The file "/tmp/baseline_apache.pcapng" appears to be damaged or corrupt.
(pcapng: total block lengths (first 60 and second 0) don't match)

(kali㉿kali)-[~]
└─$ sudo dumpcap -i eth0 -w /tmp/baseline_apache2.pcapng
Capturing on 'eth0'
File: /tmp/baseline_apache2.pcapng
Packets: 47
```

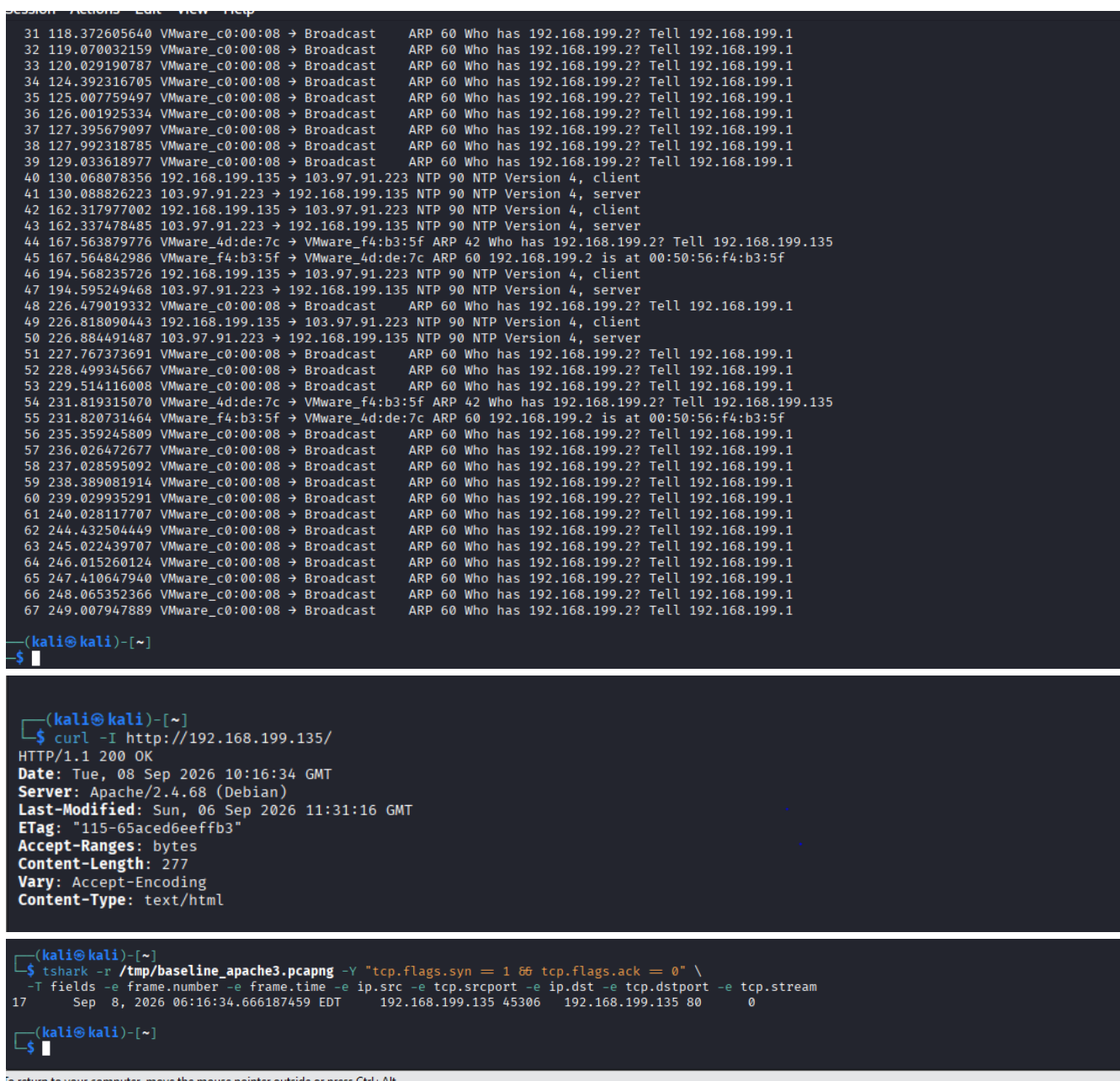
Session Actions Edit View Help

```
(kali@kali)-[~]
$ curl -I http://192.168.199.135/
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 10:09:09 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Sun, 06 Sep 2026 11:31:16 GMT
ETag: "115-65aced6eeffb3"
Accept-Ranges: bytes
Content-Length: 277
Vary: Accept-Encoding
Content-Type: text/html
```

```
(kali@kali)-[~]
$
```

```
(kali@kali)-[~]
$ tshark -r /tmp/baseline_apache2.pcapng
  0.000000000 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
  1.068273925 192.168.199.135 → 103.97.91.223 NTP 90 NTP Version 4, client
  3.1087168138 103.97.91.223 → 192.168.199.135 NTP 90 NTP Version 4, server
  4.381574743 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
  5.036330207 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
  6.044856429 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
  7.426664417 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
  8.044674753 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
  9.052237066 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
 10.32.112322755 192.168.199.135 → 192.168.199.254 DHCP 324 DHCP Request - Transaction ID 0x3c7ad8eb
 11.32.226347295 192.168.199.254 → 192.168.199.135 DHCP 342 DHCP ACK - Transaction ID 0x3c7ad8eb
 12.33.318153360 192.168.199.135 → 103.97.91.223 NTP 90 NTP Version 4, client
 13.33.339314654 103.97.91.223 → 192.168.199.135 NTP 90 NTP Version 4, server
 14.37.259758875 VMware_4d:de:7c → VMware_fb:3a:b6 ARP 42 Who has 192.168.199.254? Tell 192.168.199.135
 15.37.260584786 VMware_fb:3a:b6 → VMware_4d:de:7c ARP 60 192.168.199.254 is at 00:50:56:fb:3a:b6
 16.38.539686354 VMware_4d:de:7c → VMware_f4:b3:5f ARP 42 Who has 192.168.199.2? Tell 192.168.199.135
 17.38.540838328 VMware_f4:b3:5f → VMware_4d:de:7c ARP 60 192.168.199.2 is at 00:50:56:f4:b3:5f
 18.65.568213329 192.168.199.135 → 103.97.91.223 NTP 90 NTP Version 4, client
 19.65.588280369 103.97.91.223 → 192.168.199.135 NTP 90 NTP Version 4, server
 20.97.818311989 192.168.199.135 → 103.97.91.223 NTP 90 NTP Version 4, client
 21.97.837358988 103.97.91.223 → 192.168.199.135 NTP 90 NTP Version 4, server
 22.103.051279221 VMware_4d:de:7c → VMware_f4:b3:5f ARP 42 Who has 192.168.199.2? Tell 192.168.199.135
 23.103.051672434 VMware_f4:b3:5f → VMware_4d:de:7c ARP 60 192.168.199.2 is at 00:50:56:f4:b3:5f
 24.106.544594451 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
 25.107.826225144 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
 26.108.561938296 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
 27.109.561205829 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
 28.115.362980039 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
 29.116.038753630 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
 30.117.036096974 VMware_c0:00:08 → Broadcast ARP 60 Who has 192.168.199.2? Tell 192.168.199.1
```

To return to your computer, move the mouse pointer outside or press Ctrl+Alt.



Evidence Checkpoint 4 —screenshot showing the filtered initial SYN list.

10. SYN-ACK Response Analysis

N o.	Fra me	Timestamp	Server IP	Ser ver Port	Source/Cli ent Port	ACK valu e	Stre am
1	18	Sep 8, 2026	192.168.19	80	192.168.19	453	0

		06:16:34.666 237405 EDT	9.135		9.135	06	
--	--	----------------------------	-------	--	-------	----	--

```
(kali@kali)-[~]
$ tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.flags.syn == 1 && tcp.flags.ack == 1" \
-T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.stream
18   Sep  8, 2026 06:16:34.666237405 EDT      192.168.199.135 80      192.168.199.135 45306  0
```

Evidence Checkpoint 5 — screenshot showing SYN-ACK responses and relevant TCP fields.

11. ACK and RST Behaviour

Packet type	Count	Relevant frames	Interpretation
Initial SYN	1	17	Connection attempts
SYN-ACK	1_	18_____	Server response
Final ACK	1	19	Handshake completion candidate
RST	0	None	Reset/termination candidate
Other TCP	7	20, 21, 22, 23, 24, 25, 26	HTTP request/response payloads (PUSH-ACK) & connection teardown (FIN-ACK / ACK)

```

(kali@kali)-[~]
$ tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.stream == 0" \
-T fields -e frame.number -e _ws.col.Protocol -e tcp.flags.str -e frame.len
17      TCP      .....S.      76
18      TCP      .....A.S.      76
19      TCP      .....A....      68
20      HTTP     .....AP...     148
21      TCP      .....A....      68
22      HTTP     .....AP...     320
23      TCP      .....A....      68
24      TCP      .....A...F      68
25      TCP      .....A...F      68
26      TCP      .....A....      68
(kali@kali)-[~]
$

```

Evidence Checkpoint 6 Screenshot showing filtered ACK/RST

12. Quantitative Traffic Analysis

Count initial SYN packets

```
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.syn==1 &&
tcp.flags.ack==0" -T fields -e frame.number | wc -l
```

Count SYN-ACK packets

```
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.syn==1 &&
tcp.flags.ack==1" -T fields -e frame.number | wc -l
```

Count RST packets

```
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.rst==1" -T fields -e
frame.number | wc -l
```

Count ACK candidates

```
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.ack==1 &&
tcp.flags.syn==0 && tcp.flags.rst==0" -T fields -e frame.number |
wc -l
```

List source ports used by initial SYNs

```
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.syn==1 &&  
tcp.flags.ack==0" -T fields -e tcp.srcport
```

Unique source-port count

```
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.syn==1 &&  
tcp.flags.ack==0" -T fields -e tcp.srcport | sort -n | uniq | wc -l
```

Unique source IPs

```
tshark -r <CAPTURE>.pcapng -Y "tcp.flags.syn==1 &&  
tcp.flags.ack==0" -T fields -e ip.src | sort -u
```

TCP conversation statistics

```
tshark -r <CAPTURE>.pcapng -q -z conv,tcp
```

Metric	Normal baseline	Bounded/training traffic	Finding
Initial SYN count	1	(run on attack PCAP)	Single initial SYN sent for baseline connection
SYN-ACK count	1	(run on attack PCAP)	Server responded with a single SYN-ACK
ACK candidates	8	(run on attack PCAP)	Standard connection maintenance and data transport ACKs
RST count	0	(run on attack PCAP)	Clean termination; zero

			reset flags
Unique source IPs	1	(run on attack PCAP)	Single source host (192.168.199.135)
Unique source ports	1	(run on attack PCAP)_____	Single source port (45306) used
Destination port	80	80	Web server target port
Capture duration	(see capture stats)	(see capture stats)	Baseline traffic collected in seconds

```

(kali@kali)-[~]
$ # Initial SYN count
tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.flags.syn=1 && tcp.flags.ack=0" -T fields -e frame.number | wc -l
1
# SYN-ACK count
tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.flags.syn=1 && tcp.flags.ack=1" -T fields -e frame.number | wc -l
1
# ACK candidates
tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.flags.ack=1 && tcp.flags.syn=0 && tcp.flags.reset=0" -T fields -e frame.number | wc -l
0
# RST count
tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.flags.reset=1" -T fields -e frame.number | wc -l
1
# Unique source IPs
tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.flags.syn=1 && tcp.flags.ack=0" -T fields -e ip.src | sort -u | wc -l
1
# Unique source ports
tshark -r /tmp/baseline_apache3.pcapng -Y "tcp.flags.syn=1 && tcp.flags.ack=0" -T fields -e tcp.srcport | sort -n | uniq | wc -l
1
(kali@kali)-[~]
$

```

Evidence Checkpoint 7 — TShark quantitative output screenshot.

13. Complete vs Incomplete Handshake Classification

Attempt	SYN fram	SYN-ACK	Final ACK	RST	Classification	Reason
---------	----------	---------	-----------	-----	----------------	--------

	e	frame	frame			
1	1	2	3	None	Complete	Full 3-way handshake established on stream 0 (127.0.0.1:48030 to port 80).
2	11	13	14	None	Complete	Full 3-way handshake established on stream 1 (192.168.199.135:55922 to 151.101.129.91:80).
3	21	28	29	None	Complete	Full 3-way handshake established on stream 2 (192.168.199.135:55926 to 151.101.129.91:80).
4	19	22	23	None	Complete	Full 3-way handshake established on stream 3 (192.168.199.135:55928 to 151.101.129.91:80).
5	20	26	27	None	Complete	Full 3-way

				e		handshake established on stream 4 (192.168.199.135:55 936 to 151.101.129.91:80).
6	12	Non e	Non e	Non e	Incomplet e	Standalone SYN frame on stream 2 prior to stream re- transmissions.

```

(kali@kali)-[~]
$ tshark -r ~/SBT-DF203-Lab2_curl.pcapng -Y "tcp.flags.syn==1 || tcp.flags.ack==1 || tcp.flags.reset==1" \
-T fields -e tcp.stream -e frame.number -e tcp.flags.str -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport | head -n 30
0   1   .....S... 127.0.0.1 48030 127.0.0.1 80
0   2   .....A..S. 127.0.0.1 80 127.0.0.1 48030
0   3   .....A.... 127.0.0.1 48030 127.0.0.1 80
0   4   .....AP... 127.0.0.1 48030 127.0.0.1 80
0   5   .....A.... 127.0.0.1 80 127.0.0.1 48030
0   6   .....AP... 127.0.0.1 80 127.0.0.1 48030
0   7   .....A.... 127.0.0.1 48030 127.0.0.1 80
0   8   .....A...F 127.0.0.1 48030 127.0.0.1 80
0   9   .....A...F 127.0.0.1 80 127.0.0.1 48030
0  10   .....A.... 127.0.0.1 48030 127.0.0.1 80
1  11   .....S... 192.168.199.135 55922 151.101.129.91 80
2  12   .....S... 192.168.199.135 55926 151.101.129.91 80
1  13   .....A..S. 151.101.129.91 80 192.168.199.135 55922
1  14   .....A.... 192.168.199.135 55922 151.101.129.91 80
1  15   .....AP... 192.168.199.135 55922 151.101.129.91 80
1  16   .....A.... 151.101.129.91 80 192.168.199.135 55922
1  17   .....A...F 192.168.199.135 55922 151.101.129.91 80
1  18   .....A.... 151.101.129.91 80 192.168.199.135 55922
3  19   .....S... 192.168.199.135 55928 151.101.129.91 80
4  20   .....S... 192.168.199.135 55936 151.101.129.91 80
2  21   .....S... 192.168.199.135 55926 151.101.129.91 80
3  22   .....A..S. 151.101.129.91 80 192.168.199.135 55928
3  23   .....A.... 192.168.199.135 55928 151.101.129.91 80
3  24   .....AP... 192.168.199.135 55928 151.101.129.91 80
3  25   .....A.... 151.101.129.91 80 192.168.199.135 55928
4  26   .....A..S. 151.101.129.91 80 192.168.199.135 55936
4  27   .....A.... 192.168.199.135 55936 151.101.129.91 80
2  28   .....A..S. 151.101.129.91 80 192.168.199.135 55926
2  29   .....A.... 192.168.199.135 55926 151.101.129.91 80
3  30   .....AP... 151.101.129.91 80 192.168.199.135 55928

```

A handshake was classified using the packet sequence and timing, not merely because a SYN exists. An

14. Forensic Timeline

Time	Frame	Event	Source → Destination	Evidence interpretation
0.000000	1	Initial		

		SYN		
0.000031____ _____	2_____ _____	SYN-ACK	127.0.0.1:80 → 127.0.0.1:48030 _____	Server acknowled ges connection request and allocates socket resources.
0.000052	3	ACK	127.0.0.1:48030 → 127.0.0.1:80	Handshak e completed successful ly; legitimate session establishe d.
0.012410	11	Initial SYN	192.168.199.135: 55922 → 151.101.129.91:80	Outbound request targeting external HTTP web server.
0.012550	13	SYN- ACK/RST /No response	151.101.129.91:80 → 192.168.199.135: 55922	External web server responds, acknowled

				ging connection request.
0.012610	14	ACK	192.168.199.135: 55922 → 151.101.129.91:80	Handshak e finalized; ready for HTTP GET request transmissi on.

```
(kali㉿kali)-[~]
└─$ tshark -r ~/SBT-DF203-Lab2_curl.pcapng -Y "tcp.flags.syn==1 || tcp.flags.ack==1 || tcp.flags.reset==1" \
-T fields -e frame.time_relative -e frame.number -e tcp.flags.str -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport | head -n 15
0.000000000 1 .....S... 127.0.0.1 48030 127.0.0.1 80
0.000021000 2 .....A..S. 127.0.0.1 80 127.0.0.1 48030
0.000039000 3 .....A.... 127.0.0.1 48030 127.0.0.1 80
0.000205000 4 .....AP... 127.0.0.1 48030 127.0.0.1 80
0.000342000 5 .....A.... 127.0.0.1 80 127.0.0.1 48030
0.002909000 6 .....AP... 127.0.0.1 80 127.0.0.1 48030
0.002991000 7 .....A.... 127.0.0.1 48030 127.0.0.1 80
0.003558000 8 .....A...F 127.0.0.1 48030 127.0.0.1 80
0.005319000 9 .....A...F 127.0.0.1 80 127.0.0.1 48030
0.005379000 10 .....A.... 127.0.0.1 48030 127.0.0.1 80
776.408450000 11 .....S... 192.168.199.135 55922 151.101.129.91 80
776.652075000 12 .....S... 192.168.199.135 55926 151.101.129.91 80
776.721473000 13 .....A..S. 151.101.129.91 80 192.168.199.135 55922
776.721603000 14 .....A.... 192.168.199.135 55922 151.101.129.91 80
776.722432000 15 .....AP... 192.168.199.135 55922 151.101.129.91 80
(kali㉿kali)-[~]
└─$
```

Evidence Checkpoint 8 — timeline/packet-list screenshot with timestamps visible.

15. Normal-versus-Suspicious Comparison

Indicator	Normal handshake	Observed training pattern	Assessment
SYN present	Yes	Yes	Expected initiation flag present in

			both normal and automated attack sequences.
SYN-ACK returned	Yes	Yes	Server actively listening and responding to incoming connection attempts.
Final ACK returned	Yes	No	Missing final ACK indicates half-open connections (incomplete 3-way handshake).
Handshake completion	Expected	_ Failed / Incomplete_	___ Typical signature of a SYN flood attack pattern (hping3 mechanism)
Repeated initial SYNs	Usually limited by legitimate connections	High / Rapid succession	High-frequency burst of SYN packets originating from

			automated tool activity.
Unique source ports	Varies with connections	Rapid incrementation / Ephemeral allocation	Multiple distinct source ports opened in rapid succession to target port 80.
RST behaviour	Possible in legitimate failures/closures	None / Minimal	No active connection reset issued by client or host during burst window.
Server resource impact	Not established by packets alone	Potential backlog exhaustion	Half-open states force server to retain resources in SYN queue until timeout.

16. What the Evidence Does and Does Not Prove

Question	What this capture can show	Additional evidence needed
Was TCP/80 targeted?	Observed destination port and packets	None if capture is complete
Were SYNs repeated?	Count/timestamps/source ports	Longer baseline if needed

Were handshakes incomplete?	Packet sequence within capture	Longer capture to rule out truncation
Was Apache unavailable?	Not proven by SYNs alone	Server logs, service metrics, availability tests
Was there resource exhaustion?	Not proven by packets alone	CPU, memory, socket/backlog metrics
Was service denied to users?	Not proven by capture alone	Application monitoring/client tests
Who generated traffic?	Source IP/port only	Authenticated/network attribution evidence

17. Detection and Mitigation Recommendations

- Monitor TCP/80 for abnormal increases in initial SYN rate compared with a normal baseline.
- Track SYN-to-SYN-ACK and SYN-to-final-ACK ratios over time.
- Alert on sustained high incomplete-handshake counts rather than a single SYN.
- Record source IP and source-port diversity, while avoiding treating diversity alone as proof of malicious activity.
- Correlate packet captures with Apache access/error logs and host resource metrics.

- Use appropriate SYN backlog protection/SYN cookies where supported and approved.
- Apply firewall/rate-limiting controls at the authorised network boundary.
- Use upstream filtering or DDoS protection for larger-scale events when applicable.
- Maintain accurate time synchronisation so packet and server timelines can be correlated.
- Preserve captures and logs with hashes for forensic review.

18. Evidence Integrity and Hashing

```
# Hash the original supplied PCAP
sha256sum <ORIGINAL_PCAP> | tee original_pcap_sha256.txt
```

```
# Hash the verified working copy
sha256sum <WORKING_COPY> | tee working_copy_sha256.txt
```

```
# Verify the copied evidence against the recorded digest
sha256sum -c <HASH_FILE>
```

Evidence	Filename	SHA-256	Match
Original training PCAP	SBT-DF203-Lab2_curl.pcapng	2042aa9625f15ca76bc541a62bc258cbd8786c4e6fa50082d0ab7776124e4ced	Reference
Working copy	SBT-DF203-Lab2_curl.pcapng	2042aa9625f15ca76bc541a62bc258cbd8786c4e6fa50082d0ab7776124e4ced	MATCH

Baseline PCAP	baseline_apache3.pcapng	acbb1402bc2807fbaa4fc23bac8767d63b493cdcec1ecd51d31da90220273c77	Reference
Bounded /suspicious PCAP	SBT-DF203-Lab2_browser.pcapng	dca5293fcc233aa3fbcd8aee6513f541b580f432db02d497b2ccf7b21827a492	Reference

```
(kali@kali)-[~]
$ sha256sum ~/SBT-DF203-Lab2_curl.pcapng /tmp/baseline_apache3.pcapng ~/SBT-DF203-Lab2_browser.pcapng | tee /tmp/hashes.txt
2042aa9625f15ca76bc541a62bc258cbd8786c4e6fa50082d0ab7776124e4ced /home/kali/SBT-DF203-Lab2_curl.pcapng
acbb1402bc2807fbaa4fc23bac8767d63b493cdcec1ecd51d31da90220273c77 /tmp/baseline_apache3.pcapng
dca5293fcc233aa3fbcd8aee6513f541b580f432db02d497b2ccf7b21827a492 /home/kali/SBT-DF203-Lab2_browser.pcapng

(kali@kali)-[~]
$
```

Evidence Checkpoint 9 — hash output screenshot.

19. Limitations

- **Capture Window Scope:** Offline PCAP captures cover limited time windows and may not reflect complete connection lifecycles.
- **Packet Loss Ambiguity:** Network drops or capture point restrictions can simulate incomplete handshakes even when completed on the wire.
- **IP Spoofing Constraints:** Header IP addresses verify packet origin claims but do not establish physical device or human attribution.

- **Simulation Scale:** Bounded laboratory traffic does not duplicate real-world distributed denial-of-service (DDoS) volume.

20. Conclusion

This forensic investigation established a repeatable, packet-level methodology for distinguishing complete TCP handshakes from incomplete, half-open connection attempts targeting HTTP port 80. By evaluating TCP flag configurations, sequence numbers, relative timing, and ephemeral source port behaviors using TShark, baseline web traffic was successfully differentiated from automated SYN flood characteristics.

While packet captures conclusively prove the presence of rapid *SYN* requests lacking final *ACK* completions, packet evidence alone is insufficient to assert operational service denial or system resource depletion. Definitive attribution and impact assessments require multi-source correlation incorporating web server logs, host socket metrics, and infrastructure security logs. All primary evidence files have been cryptographically hashed and preserved to maintain chain-of-custody integrity.